



Kernel optimizations





Advice for kernel optimizations

- ▶ During these tests, you will make countless kernel updates, and will have to test them on the hardware to make sure nothing is broken.
- ▶ During this phase, we recommend to switch to loading the kernel through the network (`tftp`) if possible. This will save a lot of time compared to updating the kernel on the SD card, and reduces the risk to damage your SD card reader.
- ▶ Loading the kernel through the network will introduce delays and jitter, but that won't be an issue:
 - In the cases when what you measure is kernel size reduction, just making sure each new kernel is still functional.
 - If you want to measure the boot time impact of your changes, you can still start counting time from the `Starting kernel` message.
- ▶ Make kernel configuration changes **very progressively** and keep manual snapshots of each configuration. This will help when a change breaks a working kernel.



Measure - Kernel initialization functions

To find out which kernel initialization functions are the longest to execute, add `initcall_debug` to the kernel command line. Here's what you get on the kernel log:

```
...
[ 3.750000] calling  ov2640_i2c_driver_init+0x0/0x10 @ 1
[ 3.760000] initcall ov2640_i2c_driver_init+0x0/0x10 returned 0 after 544 usecs
[ 3.760000] calling  at91sam9x5_video_init+0x0/0x14 @ 1
[ 3.760000] at91sam9x5-video f0030340.lcdhe01: video device registered @ 0xe0d3e340, irq = 24
[ 3.770000] initcall at91sam9x5_video_init+0x0/0x14 returned 0 after 10388 usecs
[ 3.770000] calling  gspca_init+0x0/0x18 @ 1
[ 3.770000] gspca_main: v2.14.0 registered
[ 3.770000] initcall gspca_init+0x0/0x18 returned 0 after 3966 usecs
...
```

You might need to increase the log buffer size with `CONFIG_LOG_BUF_SHIFT` in your kernel configuration. You will also need `CONFIG_PRINTK_TIME` and `CONFIG_KALLSYMS` .



Kernel boot graph

With `initcall_debug`, you can generate a boot graph making it easy to see which kernel initialization functions take most time to execute.

- ▶ Copy and paste the output of the `dmesg` command to a file (let's call it `boot.log`)
- ▶ On your workstation, run the `scripts/bootgraph.pl` script in the kernel sources:
`scripts/bootgraph.pl boot.log > boot.svg`
- ▶ You can now open the boot graph with a vector graphics editor such as `inkscape`:

0.23 0.34 0.46 0.57 0.69 0.8 0.92 1.03 1.15 1.26 1.38 1.49 1.61 1.72 1.84



Using the kernel boot graph (1)

Start working on the functions consuming most time first. For each function:

- ▶ Look for its definition in the kernel source code. You can use Elixir (for example <https://elixir.bootlin.com>).
- ▶ Be careful: some function names don't exist, the names correspond to *modulename_init*. Then, look for initialization code in the corresponding module.
- ▶ Remove unnecessary functionality:
 - Find which kernel configuration parameter compiles the code, by looking at the Makefile in the corresponding source directory.



Using the kernel boot graph (2)

- ▶ Postpone:
 - Find which module (if any) the function belongs to. Load this module later if possible.
- ▶ Optimize necessary functionality:
 - Look for parameters which could be used to reduce probe time, looking for the `module_param` macro.
 - Look for delay loops and calls to functions containing `delay` in their name, which could take more time than needed. You could reduce such delays, and see whether the code still works or not.



Reduce kernel size

First, we focus on reducing the size without removing features

- ▶ The main mechanism is to use kernel modules
- ▶ Compile everything that is not needed at boot time as a module
- ▶ Two benefits: the kernel will be smaller and load faster, and less initialization code will get executed
- ▶ Remove features that are not used by userland: `CONFIG_KALLSYMS` , `CONFIG_DEBUG_FS` , `CONFIG_BUG`
- ▶ Use features designed for embedded systems: `CONFIG_EMBEDDED` , `CONFIG_SLUB_TINY` (reducing memory footprint for systems with less than 16MB of RAM, but not scaling well).



Reduce kernel size - Detect the biggest symbols

- ▶ Use this command to find the biggest symbols in the compiled kernel: `nm --size -r vmlinux`
- ▶ For those which could be unnecessary, look for them in the code
- ▶ Then study the corresponding `Makefile` to see how not to compile them, if possible.
- ▶ See https://elinux.org/System_Size

```
$ nm --size -r vmlinux
00003f00 b serial8250_ports
000039c0 D v4l2_dv_timings_presets
000038b8 T hidinput_connect
00003790 d edid_cea_modes_1
00002680 d drm_dmt_modes
00002000 b page_address_maps
00002000 d crc32table_le
00002000 d crc32table_be
00002000 d crc32ctable_le
00002000 d blake2s_testvecs
00001b90 b fb_display
00001b0a T v4l2_ctrl_get_name
00001ae8 t usbdev_ioctl
00001ac0 t v4l_enum_fmt
000019e0 t do_con_write
...
```




Kernel Compression

Depending on the balance between your storage reading speed and your CPU power to decompress the kernel, you will need to benchmark different compression algorithms. Also recommended to experiment with compression options at the end of the kernel optimization process, as the results may vary according to the kernel size.

.config - Linux/arm 5.15.0-rc6 Kernel Configuration

Kernel compression mode

Default mode →

Gzip
<X> LZMA
XZ
LZ0
LZ4

← Good balance between compression and speed

← Very good compression rate but slow

← Best compression rate but slow

← Poor compression rate but fast decompression

← Poorest compression rate but fastest decompression



Kernel compression options

Results on TI AM335x (ARM), 1 GHz, Linux 5.1

Timestamp	gzip	lzma	xz	lzo	lz4
Size	2350336	1777000	1720120	2533872	2716752
Copy	0.208 s	0.158 s	0.154 s	0.224 s	0.241 s
Time to userspace	1.451 s	2.167 s	1.999s	1.416 s	1.462 s

Gzip is close. It's time to try with faster storage (SanDisk Extreme Class A1)

Timestamp	gzip	lzma	xz	lzo	lz4
Size	2350336	1777000	1720120	2533872	2716752
Copy	0.150 s	0.114 s	0.111 s	0.161 s	0.173 s
Time to userspace	1.403 s	2.132 s	1.965 s	1.363 s	1.404 s

Lzo and Gzip seem the best solutions. Always benchmark as the results depend on storage and CPU performance.



Compressing the kernel with Zstandard

- ▶ Zstandard is a relatively recent compression scheme, implemented by Yann Collet.
- ▶ Unfortunately, not available on ARM yet. Only on x86, mips and s390 (Linux 6.4 status).
- ▶ Compressing better than gzip and decompressing as fast as LZO, it could be the best option.
- ▶ See <https://en.wikipedia.org/wiki/Zstandard>

```
config KERNEL_ZSTD
    bool "ZSTD"
    depends on HAVE_KERNEL_ZSTD
    help
        ZSTD is a compression algorithm targeting intermediate compression
        with fast decompression speed. It will compress better than GZIP and
        decompress around the same speed as LZ0, but slower than LZ4. You
        will need at least 192 KB RAM or more for booting. The zstd command
        line tool is required for compression.
```



Booting an uncompressed kernel

- ▶ It is also possible to boot an uncompressed kernel: [arch/<arch>/boot/Image](#)
- ▶ This could be a worthy solution if you have a slow CPU and fast I/O, or if you're booting Linux in an emulated machine (hardware or software emulator).
- ▶ On U-Boot on ARM, you won't be able to boot with the [bootz](#) command. You will need to use [bootm](#) and a [uImage](#) file.

See <https://bootlin.com/blog/uncompressed-linux-kernel-on-arm/>



Optimize kernel for size (1)

- ▶ `CONFIG_CC_OPTIMIZE_FOR_SIZE` : possibility to compile the kernel with `gcc -Os` instead of `gcc -O2`.
- ▶ Such optimizations give priority to code size at the expense of code speed. `-Os` enables all `-O2` optimizations except those that often increase code size.
- ▶ Results: loading and decompressing the kernel is faster (smaller size), but then the kernel boots and runs slower.



Optimize kernel for size (2)

Results on BeagleBone Black, Linux 5.11, lzo compression

	O2	Os	Diff
Size	7372432	6594440	−10.5 %
Copy time	0.489 s	0.437s s	−52 ms
Decompression time	1.490 s	1.558 s	−68 ms
Time to userspace	1.303 s	1.462 s	+159 ms
Total boot time	5.739 s	5.796s	+57 ms



Deferring drivers and initcalls

- ▶ If you can't compile a feature as a module (e.g. networking or block subsystem), you can try to defer its execution.
- ▶ Your kernel will not shrink but some initializations will be postponed.
- ▶ Typically, you would modify `probe()` functions to return - `EPROBE_DEFER` until they are ready to be run.
- ▶ See <https://lwn.net/Articles/485194/> for details about the infrastructure supporting this.



Turning off console output

- ▶ Console output is actually taking a lot of time (very slow device). Probably not needed in production. Disable it by passing the `quiet` argument on the kernel command line.
- ▶ You will still be able to use `dmesg` to get the kernel messages.
- ▶ Time between starting the kernel and starting the `init` program, on Microchip SAMA5D3 Xplained (ARM), Linux 3.10:

	Time	Diff
Without <code>quiet</code>	2.352 s	
With <code>quiet</code>	1.285 s	−1.067 s

- ▶ Less time will be saved on a reduced kernel, of course.
- ▶ Don't do it too early if you're using `grabserial`



Preset loops per jiffy

- ▶ At each boot, the Linux kernel calibrates a delay loop (for the `udelay()` function). This measures a number of loops per jiffy (*lpj*) value. You just need to measure this once! Find the `lpj` value in the kernel boot messages:

```
Calibrating delay loop... 996.14 BogoMIPS (lpj=4980736)
```

- ▶ Now, you can add `lpj=<value>` to the kernel command line:

```
Calibrating delay loop (skipped) preset value.. 996.14 BogoMIPS (lpj=4980736)
```

- ▶ Tests on BeagleBone Black (ARM), Linux 6.1: —83 ms



Multiprocessing support (CONFIG_SMP)

- ▶ SMP is quite slow to initialize
- ▶ It is usually enabled in default configurations, even if you have a single core CPU (default configurations should support multiple systems).
- ▶ So make sure you disable it if you only have one CPU core.
- ▶ Results on BeagleBone Black: Compressed kernel size: —188 KB



Kernel: last milliseconds (1)

To shave off the last milliseconds, you will probably want to remove unnecessary features:

- ▶ `CONFIG_PRINTK=n` will have the same effect as the `quiet` command line argument but you won't have any access to kernel messages. You will have a significantly smaller kernel though.
- ▶ Compile your kernel in *Thumb2* mode (on ARM 32 bit):
`CONFIG_THUMB2_KERNEL` (any ARM toolchain can do that).



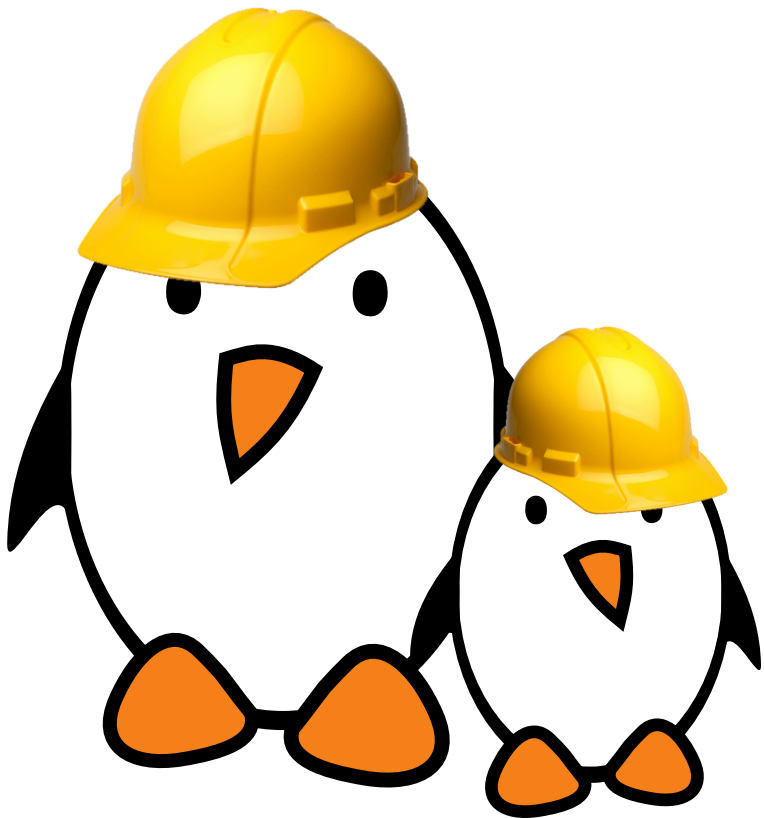
Kernel last milliseconds (2)

More features you could remove:

- ▶ Module loading/unloading
- ▶ Block layer
- ▶ Network stack
- ▶ USB stack
- ▶ Power management features
- ▶ `CONFIG_SYSFS_DEPRECATED`
- ▶ Input: keyboards / mice / touchscreens



Practical lab - Reduce kernel boot time



- ▶ Use `initcall_debug` to find the biggest time consumers
- ▶ Optimize existing functionality
- ▶ Remove unused features and drivers
- ▶ Select the best kernel compression method